Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Event-Driven Mail Processing with Microsoft Graph Change Notifications 5

What this solution is . . . 5

Why event-driven (and why not the O365 connector or a poll) . . . 5

How it works . . . 6

What you control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications 7

2.1 An Azure Logic App with a reachable public HTTPS notificationUrl . . . 8

2.2 A managed identity on the Logic App . . . 8

2.3 A least-privilege application Graph permission, admin-consented . . . 8

2.4 Mailbox access scoped to the one target mailbox . . . 8

2.5 An Exchange Online target mailbox . . . 9

2.6 The Microsoft.Graph PowerShell module . . . 9

2.7 A secret store for clientState (and any encryption key) . . . 9

2.8 TLS and endpoint trust . . . 9

2.9 National-cloud endpoints (if applicable) . . . 10

Checklist . . . 10

Global Requirements — Azure Solutions 10

G1. A Microsoft 365 / Entra tenant . . . 10

G2. Administrative roles (least privilege) . . . 11

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 11

G5. Change control & maintenance window . . . 12

G6. Backup / rollback posture . . . 12

3. Implementation Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications 12

Naming (set once, reuse verbatim) . . . 12
Step 1 — Enable the managed identity on the Logic App . . . 13
Step 2 — Grant the least-privilege application permission + admin consent . . . 13
Step 3 — Scope mailbox access to the one target mailbox (concise — defer depth to product #4) . . . 14
Step 4 — Store the clientState secret . . . 14
Step 5 — Build the receiver workflow (`wf-NotificationReceiver`) . . . 14
Step 6 — Create the change-notification subscription . . . 15
Step 7 — Build the renewal + catch-up workflow (`wf-SubscriptionRenewal`) . . . 16
Step 8 — Reliability: dedup, retry, dead-letter, exactly-once intent . . . 16
Step 9 — Pilot, validate, expand . . . 17
5. Testing Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications 17
Test 1 — Validation handshake succeeds (subscription creates) . . . 17
Test 2 — A new message fires a notification . . . 17
Test 3 — clientState rejects a forged call . . . 18
Test 4 — Renewal extends the subscription . . . 18
Test 5 — Dedup prevents double-processing . . . 18
Test 6 — Catch-up recovers a missed event . . . 18
Test 7 — Least-privilege scoping holds . . . 18
Pass/fail gate . . . 19
6. Troubleshooting Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications 19
A. Subscription creation fails . . . 19
B. Validation handshake fails . . . 20
C. Notifications stop arriving (expired subscription) . . . 20
D. 403 when reading the changed message . . . 20
E. Duplicate processing . . . 21
F. clientState mismatch . . . 21
G. Scripts — Microsoft.Graph connection and matching . . . 21
H. When to escalate . . . 22
7. Best Practices — Event-Driven Mail Processing with Microsoft Graph Change Notifications 22
8. Deliverables Summary — Event-Driven Mail Processing with Microsoft Graph Change Notifications 23
Documentation . . . 23
PowerShell scripts (Microsoft.Graph) . . . 24
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)

White-label · Azure Solutions

Polling a mailbox on a timer is slow and wasteful, and the Office 365 Outlook connector **cannot use a managed identity** — so it drags a stored service-account credential along with it. The event-driven answer is a **Microsoft Graph change-notification (webhook) subscription**: the moment a message arrives in the target mailbox, Graph POSTs a notification to your Logic App, which reads the message and processes it — in near real time, with **no polling and no stored credentials**.

This is a complete, **deploy-it-today** package for standing up that pattern end to end, written for an administrator with no prior Graph-subscription experience. It authenticates with the Logic App's **managed identity** holding a **least-privilege application permission scoped to one mailbox** — no service account, no secret, no O365 connection.

What you get

- A plain-language **Solution Overview** of the subscription + receiver architecture, why event-driven beats the connector and the poll, and where it fits alongside the parent product (#1), the polling alternative (#3), and the mailbox-scoping deep-dive (#4).
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite — the public HTTPS endpoint, managed identity, application permission + admin consent, target mailbox, Microsoft.Graph module, TLS, and national-cloud endpoints.
- An **Implementation Guide** of precise, ordered steps — enable the identity, grant and consent the least-privilege permission, scope access to the one mailbox, build the receiver with the **validation handshake** and **clientState** check, create the subscription, and build the **renewal + catch-up** workflow.
- Five **PowerShell scripts** (Microsoft.Graph) for subscription/permission discovery, PASS/FAIL validation, a subscription inventory report with CSV, a `-WhatIf`-gated renewal helper, and a notification-evidence checker.
- **Testing** and **Troubleshooting** guides for a confident, durable rollout.

What this delivers

A **Graph change-notification subscription** on the target mailbox, a **Logic App receiver** that answers Graph's validation handshake, verifies `clientState`, and reads the changed message via **managed identity** — plus the operational hard parts most guides skip: **subscription renewal before expiry, missed-notification recovery via a catch-up read, and dedup/idempotency** so a duplicate or dropped notification never means double-processing or lost mail. Mailbox access is **least-privilege, scoped to one mailbox**.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with an explicit end-to-end event test and a documented rollback (delete the subscription; the mailbox is untouched).

Outcome

Near-real-time, credential-free mail processing that stays reliable — instead of poll latency, a stored service account, and subscriptions that silently expire — deployable by a single administrator and validated on a pilot mailbox before you scale.

*Version-specific Graph details — maximum subscription lifetime, renewal cadence, supported resources, permission names, encrypted-payload rules, national-cloud endpoints, and Consumption-vs-Standard hosting — are flagged throughout to **verify against current Microsoft documentation** rather than asserted.*

1. Solution Overview — Event-Driven Mail Processing with Microsoft Graph Change Notifications

What this solution is

This solution processes new mail in a target mailbox **the moment it arrives**, using a **Microsoft Graph change-notification (webhook) subscription** to drive an **Azure Logic App** — with **no polling timer** and **no Office 365 Outlook connector**. When a message lands in the monitored mailbox, Microsoft Graph sends an HTTP POST ("notification") to the Logic App's public HTTPS endpoint; the Logic App validates the call, then reads the changed message from Graph and runs whatever processing you need (parse, route, ticket, store).

Authentication uses the Logic App's **managed identity** holding a least-privilege **application** Graph permission (for example `Mail.Read`), and mailbox access is **scoped to the one target mailbox**. There is **no service account, no stored password, and no client secret** anywhere in the workflow.

This is the **event-driven** member of a family of related products. It is the pattern named as the "push" alternative in the parent product **"Exchange Online Mailbox Automation with Logic Apps Managed Identity — Replacing the O365 Connector with Microsoft Graph"** (product #1, the root-cause explanation of *why* the O365 connector is being retired for Graph + managed identity). Where a public HTTPS endpoint that Graph can reach is not available, use the **polling** alternative instead — **"Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)"** (product #3). For the deep treatment of **restricting an application permission to a single mailbox**, use **"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online"** (product #4); this guide performs the scoping concisely and defers the depth there.

Why event-driven (and why not the O365 connector or a poll)

- **The Office 365 Outlook connector cannot use a managed identity.** It authenticates with a delegated user connection (a service mailbox sign-in / API connection), which means a stored credential and an interactive consent that expires — exactly the dependency this family removes. *(Whether any specific connector trigger has since gained a managed-identity option is version-specific — **verify against current Microsoft documentation**.)*

• **Polling is slow and wasteful.** A timer-triggered "check the mailbox every N minutes" workflow adds latency (up to the poll interval) and burns runs even when nothing arrived. It is the right answer only when a webhook cannot be delivered — hence product #3.

• **Change notifications are near real time and efficient.** Graph pushes a small notification the instant the mailbox changes; the workflow only runs when there is actually work to do.

How it works

1. A **Graph change-notification subscription** is created against the **target mailbox's message resource** (for example `users/svc-mailevents@contoso.com/messages`), with a `changeType` of `created`, a `notificationUrl` pointing at the Logic App's HTTPS endpoint, a random secret `clientState`, and an `expirationDateTime`. *(The exact supported resource forms, the maximum allowed subscription lifetime, and the renewal cadence are Graph-version-specific — **verify against current Microsoft documentation**; never hard-code a lifetime.)*

2. **Graph validates the endpoint** at creation time: it sends a request carrying a `validationToken`, and the endpoint must **echo that token back** as `text/plain` within a few seconds or the subscription is rejected. The **receiver Logic App** answers this handshake first.

3. When a new message arrives, **Graph POSTs a notification** to the `notificationUrl`. By default the notification carries only a **reference** to the changed item (mailbox + message id + `changeType`), **not the message body**. *(Graph can optionally include **resource data** in the payload, which for mail must be **encrypted** with a key you supply — availability and encryption requirements are version-specific; **verify against current Microsoft documentation**.)*

4. The **receiver workflow verifies** `clientState` matches the secret it set, discards the call if it does not, then **reads the changed message from Graph using the managed identity** (`GET /users/{mailbox}/messages/{id}`) and processes it **idempotently**.

5. A separate **renewal workflow** runs on a schedule and **extends the subscription before it expires**, so notifications never stop. A **reconcile/catch-up read** covers any gap (a missed notification or a lapsed subscription) by reading messages since the last processed watermark.

What you control

• **Which mailbox** is monitored (one mailbox, least-privilege scoped — see product #4).

• **Which change** fires the workflow (`created` for new mail; other `changeType` values are possible — verify support per resource).

• **What the workflow does** with each message once read.

• **Subscription lifetime and renewal cadence** (within Graph's current limits — verify).

• **Idempotency and reliability** — how duplicates are suppressed and how missed events are recovered.

Benefits

• **Near-real-time** processing instead of poll-interval latency.

• **No stored credentials** — managed identity + application permission; nothing to rotate or leak.

• **Least privilege** — one application permission, scoped to one mailbox.

• **Efficient** — the workflow runs only when the mailbox actually changes.

• **Durable** — renewal and catch-up mean a single missed notification or expired subscription does not silently drop mail.

Limitations (be honest with buyers)

• **A public HTTPS endpoint is required.** Graph must be able to reach the `notificationUrl` from the internet over TLS. If your Logic App can only run on a private endpoint with no public path, use the **polling** pattern (product #3) instead.

• **Subscriptions expire and must be renewed.** There is no "permanent" subscription; the renewal workflow is not optional. A missed renewal stops notifications until re-created — the catch-up read is what prevents data loss in that window.

• **Notifications can be missed or delivered more than once.** Delivery is best-effort and at-least-once, so the workflow **must** be idempotent and must reconcile gaps; treat every notification as "maybe already processed, maybe one of several."

• **Notifications are references, not content, by default.** The workflow reads the message separately (encrypted resource-data delivery is an option to verify).

• **Graph specifics change.** Maximum lifetime, renewal cadence, supported resources, permission names, and national-cloud endpoints are all version- and cloud-specific — this guide flags each; **verify against current Microsoft documentation** before you rely on a number.

• **Consumption vs Standard Logic Apps differ** in hosting, networking, and pricing for an always-reachable HTTP endpoint — **verify against current Microsoft documentation** which plan fits your endpoint and scale needs.

The standard deployment model

> ***Enable the managed identity → grant the least-privilege application permission (admin consent) → scope access to the one mailbox (see #4) → build the receiver Logic App with the validation handshake and clientState check → create the change-notification subscription → build the renewal + catch-up workflow → pilot, validate, expand.*** *The repeatable pattern is* ***Assess → Configure → Pilot → Validate → Expand****, with an explicit end-to-end event test and a documented rollback (delete the subscription; the mailbox is untouched).*

2. Requirements Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named per the convention below and reused verbatim throughout.

> ***Naming (set once, reuse verbatim).*** *Azure ARM resources use lower-case hyphenated names; Exchange/Entra logical objects use* `<Purpose> – <Scope> – <Stage>`*. - Resource group:*

`rg-mailevents-pilot` - *Logic App (hosting the workflows):* `logic-mailevents-receiver-pilot` - *Receiver workflow:* `wf-NotificationReceiver` - *Renewal / catch-up workflow:* `wf-SubscriptionRenewal` - *Key Vault (for the* `clientState` *secret):* `kv-mailevents-pilot` - `clientState` *secret name:* `MailEvents-ClientState` - *Target mailbox:* `svc-mailevents@contoso.com` - *Mail-enabled security group for scoping:* `MailEvents – Scope – Pilot` - *Application Access Policy:* `MailEvents – AppAccess – Pilot`

2.1 An Azure Logic App with a reachable public HTTPS notificationUrl

Description: An Azure Logic App (`logic-mailevents-receiver-pilot`) whose HTTP-triggered workflow exposes an **HTTPS endpoint that Microsoft Graph can reach from the internet** to POST notifications and the validation request. **Why:** Change notifications are *pushed*. Graph must resolve and connect to the `notificationUrl` over public TLS, both at subscription-creation (the validation handshake) and for every notification. No public path = no webhook. **Verify:** From an external network, `curl -i https://<workflow-endpoint>` returns a TLS response (not a private/timeout). Run `GRAPHSUB-Discovery.ps1`. **Implement-if-missing:** Create the Logic App and an HTTP-request-triggered workflow with a public endpoint. **Consumption vs Standard** hosting differs in networking and pricing for an always-on HTTP endpoint — **verify against current Microsoft documentation** which plan fits; if only a private endpoint is possible, use the **polling** pattern (product #3) instead.

2.2 A managed identity on the Logic App

Description: A **managed identity** (system-assigned on `logic-mailevents-receiver-pilot`, or a user-assigned identity attached to it) that the workflows use to call Graph. **Why:** The whole point of this family is credential-free auth — the workflow authenticates to Graph as the managed identity, with no service account, secret, or O365 connection. **Verify:** Azure portal → the Logic App → **Identity** shows **System assigned = On** (or a user-assigned identity attached) and an **Object (principal) ID**. `GRAPHSUB-Discovery.ps1` reports it. **Implement-if-missing:** Turn on the system-assigned identity (or create and attach a user-assigned one). Record the identity's **Object ID** — you grant the Graph permission to it in Step 2 of the Implementation Guide.

2.3 A least-privilege application Graph permission, admin-consented

Description: An **application** Microsoft Graph permission — least privilege for reading mail (for example `Mail.Read`) — granted to the managed identity's service principal, with **admin consent**. **Why:** Application permissions are tenant-wide by default and require an admin to consent; the workflow needs read access to the changed message. Application (not delegated) permission is what lets an unattended workflow run with no signed-in user. **Verify:** Entra → **Enterprise applications** → the managed identity → **Permissions**; or run `GRAPHSUB-Validate.ps1`, which checks the app-role assignment exists. **Implement-if-missing:** Assign the app role to the managed identity's service principal and grant admin consent (Implementation Step 2). **The exact permission name and whether a narrower mail-read scope exists are version-specific — verify against current Microsoft documentation**; grant the *least* permission that lets the workflow read the message, never `Mail.ReadWrite` "to be safe."

2.4 Mailbox access scoped to the one target mailbox

Description: The application permission from 2.3 **restricted to only** `svc-mailevents@contoso.com`, using an **Application Access Policy** (`MailEvents – AppAccess – Pilot`) over a mail-enabled security group (`MailEvents – Scope – Pilot`) — or the newer Graph-RBAC-for-Exchange model. **Why:** A bare application `Mail.Read` grant can read **every** mailbox in the tenant. Scoping it to one mailbox is the security spine of this solution. **Verify:** `Test-ApplicationAccessPolicy` (Exchange Online PowerShell) returns **Granted** for the target mailbox and **Denied** for a control mailbox. See product #4 for the full procedure. **Implement-if-missing:** Create the scoping (Implementation Step 3, concise). **The choice between Application Access Policy and the newer Graph RBAC for Exchange Online, and which is current/recommended, is version-specific — verify against current Microsoft documentation.** For the deep treatment, **use product #4 "Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online" — this guide does not reproduce it.**

2.5 An Exchange Online target mailbox

Description: A live Exchange Online mailbox to monitor — `svc-mailevents@contoso.com` (a shared or user mailbox that receives the mail you want to process). **Why:** The change-notification subscription is created against this mailbox's message resource; it must exist and be licensed/enabled as required. **Verify:** Exchange admin center → **Recipients** → **Mailboxes** shows `svc-mailevents@contoso.com`; `Get-Mailbox svc-mailevents@contoso.com` succeeds. **Implement-if-missing:** Create the mailbox (shared mailboxes need no user license in most cases — **verify licensing against current Microsoft documentation**).

2.6 The Microsoft.Graph PowerShell module

Description: The **Microsoft.Graph** PowerShell module on the operator workstation, for discovery, validation, reporting, and renewal. **Why:** The five solution scripts use `Microsoft.Graph` cmdlets (`Connect-MgGraph`, `Get-MgSubscription`, `New-MgSubscription`, `Update-MgSubscription`, `Get-MgServicePrincipal`). **Verify:** `Get-Module -ListAvailable Microsoft.Graph`; `GRAPHSUB-Discovery.ps1` self-checks and installs if missing. **Implement-if-missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.7 A secret store for clientState (and any encryption key)

Description: A secret store — `kv-mailevents-pilot` **(Azure Key Vault)** — holding the random `clientState` secret (`MailEvents-ClientState`) the subscription and receiver share, plus any resource-data encryption key if you enable encrypted payloads. **Why:** `clientState` is how the receiver proves a notification really came from your subscription; it must be a secret, not in code. The Logic App reads it via its managed identity. **Verify:** Key Vault → **Secrets** shows `MailEvents-ClientState`; the Logic App's managed identity has **Key Vault Secrets User** (or an access policy) on the vault. **Implement-if-missing:** Create the vault, add the secret (a long random string), and grant the managed identity read access. **Whether you also need an encryption key depends on using encrypted resource-data delivery — verify against current Microsoft documentation.**

2.8 TLS and endpoint trust

Description: A valid, publicly trusted **TLS** certificate on the `notificationUrl` host, meeting Graph's current TLS requirements. **Why:** Graph will not deliver to an endpoint it cannot validate over TLS; the

Logic App's default `*.azurewebsites.net`/`*.logic.azure.com` endpoint is already TLS-terminated, but a custom domain/gateway in front must present a trusted certificate. **Verify:** The endpoint answers on `https://` with a valid chain (browser/`curl` shows no cert error). **Graph's minimum TLS version is version-specific — verify against current Microsoft documentation. Implement-if-missing:** Use the platform-provided HTTPS endpoint, or install a publicly trusted certificate on the fronting gateway.

2.9 National-cloud endpoints (if applicable)

Description: If the tenant is a **government / national cloud** (GCC High, DoD, or another sovereign cloud), the correct **national-cloud Graph and login endpoints** for subscriptions and token acquisition. **Why:** Commercial `graph.microsoft.com` / `login.microsoftonline.com` are **not** the endpoints in sovereign clouds; using the wrong host fails auth and subscription creation. **Verify:** Confirm the tenant's cloud and the matching endpoints. `Connect-MgGraph -Environment <cloud>` targets the right sovereign endpoint. **Implement-if-missing: The exact national-cloud Graph/login endpoints and feature parity for change notifications are cloud-specific — verify against current Microsoft documentation** and set the module `-Environment` / Logic App endpoints accordingly. Do not assume commercial hosts.

Checklist

- Logic App with a reachable public HTTPS endpoint — `GRAPHSUB-Discovery.ps1`
- Managed identity on the Logic App (Object ID recorded)
- Least-privilege application Graph permission granted + admin-consented — `GRAPHSUB-Validate.ps1`
- Mailbox access scoped to `svc-mailevents@contoso.com` (see product #4)
- Exchange Online mailbox `svc-mailevents@contoso.com` exists
- Microsoft.Graph PowerShell module installed
- `kv-mailevents-pilot` holds `MailEvents-ClientState`; MI can read it
- Public, trusted TLS on the notificationUrl host
- National-cloud endpoints confirmed (if a sovereign tenant) — *verify with Microsoft*

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications

Follow these steps **in order**. Portal work is in the Azure portal (`https://portal.azure.com`), the Entra admin center (`https://entra.microsoft.com`), and the Exchange admin center (`https://admin.exchange.microsoft.com`). Objects are named per the convention and **reused verbatim** in every later step, script, and test.

> ***Model:*** *Enable the managed identity → grant the least-privilege application permission (admin consent) → scope to the one mailbox → build the receiver with the validation handshake and clientState check → create the subscription → build the renewal + catch-up workflow → pilot, validate, expand. Keep a rollback (delete the subscription; the mailbox is untouched).*

Naming (set once, reuse verbatim)

• Resource group: `rg-mailevents-pilot`

• Logic App (Standard or Consumption — verify plan choice): `logic-mailevents-receiver-pilot`

• Receiver workflow: `wf-NotificationReceiver`

- Renewal / catch-up workflow: `wf-SubscriptionRenewal`
- Key Vault: `kv-mailevents-pilot`
- `clientState` secret: `MailEvents-ClientState`
- Target mailbox: `svc-mailevents@contoso.com`
- Mail-enabled security group (scoping): `MailEvents – Scope – Pilot`
- Application Access Policy: `MailEvents – AppAccess – Pilot`

Step 1 — Enable the managed identity on the Logic App

1. `portal.azure.com` → create (or open) the resource group `rg-mailevents-pilot` and the Logic App `logic-mailevents-receiver-pilot`. *(Consumption vs Standard changes networking and pricing for an always-reachable HTTP endpoint — **verify against current Microsoft documentation** which plan fits before you create it.)*
2. Open `logic-mailevents-receiver-pilot` → **Settings** → **Identity**.
3. On the **System assigned** tab, set **Status = On** → **Save** → **Yes**. *(Alternatively attach a user-assigned identity on the **User assigned** tab — either works; be consistent.)*
4. Copy the **Object (principal) ID** that appears — you grant the Graph permission to this in Step 2. Run `GRAPHSUB-Discovery.ps1` to confirm the identity is present.

Step 2 — Grant the least-privilege application permission + admin consent

The managed identity's service principal must hold an **application** Graph permission to read the changed message. You grant an app role to the identity's service principal and consent to it. **The exact permission name and any narrower mail-read scope are version-specific — verify against current Microsoft documentation; grant the least permission that lets the workflow read a message (for example `Mail.Read`), never a write scope.**

1. Note the **Graph service principal AppId** `00000003-0000-0000-c000-000000000000` (Microsoft Graph) — the app roles you assign come from it.
2. Assign the app role to the managed identity's service principal. This is most reliably done with Graph PowerShell (the portal cannot always assign app roles to a managed identity). Using the values you recorded in Step 1:

```
Connect-MgGraph -Scopes 'AppRoleAssignment.ReadWrite.All','Application.Read.All'
$graph = Get-MgServicePrincipal -Filter "appId eq '00000003-0000-0000-c000-000000000000'"
$mi    = Get-MgServicePrincipal -ServicePrincipalId '<managed-identity-object-id>'
$role  = $graph.AppRoles | Where-Object { $_.Value -eq 'Mail.Read' -and $_.AllowedMemberTypes -contains
New-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $mi.Id `
    -PrincipalId $mi.Id -ResourceId $graph.Id -AppRoleId $role.Id
```

*(Assigning the app role **is** the admin consent for an application permission granted this way — it requires an admin to run it. Verify the role `Value` string against current Microsoft documentation before assigning.)*

1. Confirm with **Entra** → **Enterprise applications** → `logic-mailevents-receiver-pilot` → **Permissions**, or run `GRAPHSUB-Validate.ps1`.

Step 3 — Scope mailbox access to the one target mailbox (concise — defer depth to product #4)

A bare application `Mail.Read` can read **every** mailbox. Restrict it to `svc-mailevents@contoso.com` before going further. **This is deliberately concise — for the full treatment (Application Access Policy vs the newer Graph RBAC for Exchange Online, edge cases, and validation), use product #4 "Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online." Which model is current is version-specific — verify against current Microsoft documentation.**

Using the classic **Application Access Policy** model:

1. In **Exchange Online PowerShell** (`Connect-ExchangeOnline`), create the mail-enabled security group `MailEvents – Scope – Pilot` and add **only** `svc-mailevents@contoso.com` as a member.

2. Create the access policy `MailEvents – AppAccess – Pilot` binding the managed identity's **AppId** to that group so the app can access **only** members of the group:

```
New-ApplicationAccessPolicy -AppId '<managed-identity-app-id>' `
    -PolicyScopeGroupId 'MailEvents-Scope-Pilot@contoso.com' `
    -AccessRight RestrictAccess `
    -Description 'MailEvents – AppAccess – Pilot'
```

1. Validate scoping:

```
Test-ApplicationAccessPolicy -AppId '<managed-identity-app-id>' -Identity svc-mailevents@contoso.com
Test-ApplicationAccessPolicy -AppId '<managed-identity-app-id>' -Identity someone-else@contoso.com
```

(Access-policy changes can take time to propagate — allow for it. See product #4.)

Step 4 — Store the clientState secret

1. `portal.azure.com` → `kv-mailevents-pilot` → **Secrets** → **Generate/Import** → name `MailEvents-ClientState`, value = a long random string (for example a GUID plus random bytes). This is the shared secret the subscription sets and the receiver checks.

2. **Access control (IAM)** on the vault → grant the Logic App's managed identity **Key Vault Secrets User** (or add an access policy with **Get** on secrets), so `wf-NotificationReceiver` can read the secret at runtime.

Step 5 — Build the receiver workflow (`wf-NotificationReceiver`)

This workflow must do three things, in this order: **(1) answer Graph's validation handshake, (2) verify `clientState`, (3) process notifications idempotently.**

1. In `logic-mailevents-receiver-pilot`, create a workflow named `wf-NotificationReceiver` with a **"When an HTTP request is received"** trigger. Save it once to generate the public **HTTP POST URL** — this is your `notificationUrl`. Copy it.

2. **Handle the validation handshake first.** Graph validates the endpoint by calling it with a `validationToken` query parameter and expects that exact token echoed back as `text/plain`, HTTP 200, within a few seconds. Add a **Condition** at the very top:

• **If** the trigger query contains `validationToken` →

• **Response** action: **Status Code** `200`, **Headers** `Content-Type: text/plain`, **Body** = the `validationToken` value (URL-decoded). Return immediately.

*(The endpoint must answer the handshake within Graph's timeout — **verify the exact timeout against current Microsoft documentation**; keep this branch as the first thing the workflow does so it responds fast.)*

1. **Acknowledge fast, then process.** For a real notification (no `validationToken`), respond **202 Accepted** quickly so Graph considers delivery successful, then continue processing asynchronously. Graph retries on non-2xx, which is one source of duplicates — acknowledging promptly reduces needless retries.

2. **Verify `clientState`.** Read `MailEvents-ClientState` from `kv-mailevents-pilot` (via the managed identity) and compare it to the `clientState` field on each notification in the request body. **If it does not match, discard the notification and do not process it** (log and drop). This is what stops a forged POST to your public URL from driving the workflow.

3. **Read the changed message with the managed identity.** Each valid notification carries a `resource` reference and the changed item's id (by default **no message body**). For each notification, call Graph **authenticated as the managed identity**:

• `GET https://graph.microsoft.com/v1.0/users/svc-mailevents@contoso.com/messages/{id}`

• In a Consumption Logic App, use the **HTTP** action with **Authentication = Managed identity**, Audience `https://graph.microsoft.com`. In Standard, use the HTTP action or a Graph connector bound to the identity. *(Whether the notification's `resource` value can be used directly as the request path is version-specific — **verify against current Microsoft documentation**.)*

• *(Optionally, Graph can deliver **encrypted resource data** in the notification so you skip the read — this requires supplying an encryption key and decrypting in the workflow; availability and the encryption contract are version-specific — **verify against current Microsoft documentation**.)*

1. **Make processing idempotent.** Before acting on a message, record and check its **message id** (and `changeKey`/internet message id) against a small dedup store (a table, a Key Vault-referenced list, or your downstream system's natural key). If you have already processed that id, **skip it**. This is mandatory — notifications are at-least-once and can arrive more than once. See Step 8 for the reliability details.

2. Save `wf-NotificationReceiver`.

Step 6 — Create the change-notification subscription

Create the subscription **against the target mailbox**, pointing `notificationUrl` at the receiver from Step 5. **The maximum allowed `expirationDateTime`, the supported `resource` forms, and the supported `changeType` values are Graph-version-specific — verify against current Microsoft documentation; never hard-code a lifetime.**

1. Put the receiver URL and the same `clientState` value from Key Vault into the request. Create the subscription (as the app identity, or interactively with a scope that allows it):

```
$params = @{
  changeType         = 'created'
  notificationUrl    = '<wf-NotificationReceiver HTTP POST URL>'
  resource           = 'users/svc-mailevents@contoso.com/messages'
```

```
  expirationDateTime = (Get-Date).ToUniversalTime().AddMinutes(<within Graph max — verify>).ToString('o
  clientState        = '<value of MailEvents-ClientState>'
}
New-MgSubscription -BodyParameter $params
```

1. **Graph immediately calls the validation handshake** on your endpoint (Step 5.2). If the endpoint echoes the token correctly, the subscription is created and returns an **id** and the accepted **expirationDateTime**. If not, creation fails — see Troubleshooting.

2. Record the returned **subscription id** and **expirationDateTime**. Run `GRAPHSUB-Report.ps1` to capture the subscription inventory.

Step 7 — Build the renewal + catch-up workflow (`wf-SubscriptionRenewal`)

A subscription is **not permanent** — it must be renewed before `expirationDateTime` or notifications stop.

1. In `logic-mailevents-receiver-pilot`, create a workflow `wf-SubscriptionRenewal` with a **Recurrence** trigger. Set the interval **comfortably shorter than the subscription lifetime** so there is always margin to renew before expiry (for example, if the max lifetime is measured in a few days, renew at least daily). **The safe renewal cadence depends on the current max lifetime — verify against current Microsoft documentation.**

2. On each run, the workflow (authenticated as the managed identity):

• **Lists the subscription(s)** for the mailbox and finds the one whose id you recorded.

• **Extends it** by PATCHing a new `expirationDateTime` within the allowed window:

```
Update-MgSubscription -SubscriptionId '<id>' `
  -ExpirationDateTime (Get-Date).ToUniversalTime().AddMinutes(<within Graph max — verify>).ToString('
```

• **Recreates the subscription** if it is already gone (expired/deleted) — falling through to `New-MgSubscription` with the same parameters as Step 6.

1. **Catch-up / missed-notification recovery.** In the same workflow (or a companion one), after ensuring the subscription is alive, **reconcile**: read messages in the mailbox **received since the last processed watermark** (a stored timestamp or last message id) via `GET /users/svc-mailevents@contoso.com/messages?$filter=receivedDateTime ge <watermark>` and process any not already in the dedup store. This closes the gap for anything that arrived while a subscription was lapsed or a notification was dropped. Advance the watermark after each successful reconcile.

2. **Lifecycle monitoring.** Enable Logic App run history / alerts on both workflows so a failed renewal or a run of receiver failures raises a signal. Optionally subscribe to Graph **lifecycle notifications** (reauthorization/subscription-removed events) if supported for this resource — **verify against current Microsoft documentation**.

3. Save `wf-SubscriptionRenewal`.

Step 8 — Reliability: dedup, retry, dead-letter, exactly-once intent

1. **Dedup key.** Use the message **id** (plus `changeKey`/internet message id) as the idempotency key; check it in the dedup store before processing (Step 5.6).

2. **Retry.** Configure the downstream processing action's retry policy (exponential backoff) so a transient failure retries rather than dropping the message.

3. **Dead-letter.** Route messages that fail after all retries to a **dead-letter store** (a queue or table) with the message id and error, so nothing is silently lost and it can be reprocessed.

4. **Exactly-once *intent*.** True exactly-once is not guaranteed by the transport; you achieve *effective* exactly-once by combining **at-least-once delivery + idempotent processing + a durable dedup store**. Design downstream actions to be safe to repeat.

Step 9 — Pilot, validate, expand

1. **Pilot** on `svc-mailevents@contoso.com` only. Run the full **Testing Guide** — validation handshake, a real new-message event, a forged-call rejection, a renewal, and a dedup test.

2. **Validate** with `GRAPHSUB-Validate.ps1` (identity + least-priv permission present, an active non-expired subscription on the mailbox, receiver reachable as far as Graph can be checked) and `GRAPHSUB-Report.ps1` (subscription inventory + CSV).

3. **Expand** — repeat the pattern for additional mailboxes by adding each to a scoped group and creating its own subscription (one subscription per mailbox resource). Keep one renewal workflow that iterates all monitored subscriptions.

Order of operations

`Enable MI → grant least-priv app permission (consent) → scope to one mailbox (#4) → build receiver (handshake → clientState → idempotent read) → create subscription → build renewal + catch-up → reliability → pilot → validate → expand`

5. Testing Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications

Run after the receiver and subscription are built (Implementation Steps 5–6), again after the renewal workflow (Step 7), and after each expansion. Objects are referenced by their names — `wf-NotificationReceiver`, `wf-SubscriptionRenewal`, `svc-mailevents@contoso.com`, `MailEvents-ClientState`. Each test is **PASS/FAIL**.

Test 1 — Validation handshake succeeds (subscription creates)

Do: Create the subscription (Implementation Step 6) with `New-MgSubscription`, or re-run it. Watch `wf-NotificationReceiver` run history. **Expected — PASS:** Graph calls the endpoint with a `validationToken`, the receiver echoes it back as `text/plain`/200, and `New-MgSubscription` **returns a subscription id and expirationDateTime**. **FAIL:** Creation errors (e.g. "endpoint validation failed") — the handshake branch did not answer correctly or fast enough. See Troubleshooting A/B.

Test 2 — A new message fires a notification

Do: Send an email to `svc-mailevents@contoso.com`. Watch `wf-NotificationReceiver` run history. **Expected — PASS:** Within seconds a run is triggered by a Graph POST; the workflow reads the message via the managed identity and completes its processing for that message id. **FAIL:** No run appears (subscription inactive/expired, notificationUrl wrong, or scoping/permission blocking the read) — run `GRAPHSUB-Validate.ps1` and see Troubleshooting C/D.

Test 3 — clientState rejects a forged call

Do: From an external tool, POST a fake notification body to the `notificationUrl` with a **wrong or missing** `clientState`. **Expected — PASS:** The receiver **discards** the call (no message read, no downstream processing); the run shows the clientState check failed and the notification was dropped. **FAIL:** The workflow processes the forged call — the `clientState` comparison (Implementation Step 5.4) is missing or wrong. See Troubleshooting F.

Test 4 — Renewal extends the subscription

Do: Run `wf-SubscriptionRenewal` manually (or wait for a scheduled run). Compare `expirationDateTime` before and after with `GRAPHSUB-Report.ps1`. **Expected — PASS:** The subscription's `expirationDateTime` moves **forward** (extended within Graph's allowed window); the subscription id is unchanged. If it had already expired, a new subscription is created and re-validated. **FAIL:** Expiration does not advance, or the run errors — check the identity's permission and the renewal cadence vs the current max lifetime (verify with Microsoft). See Troubleshooting C.

Test 5 — Dedup prevents double-processing

Do: Cause the same message to be delivered twice (send during a brief receiver retry, or re-POST the same valid notification body), or run the catch-up read over a message already processed by a live notification. **Expected — PASS:** The message is processed **once**; the second delivery is recognised by its message id in the dedup store and **skipped** (visible in run history). **FAIL:** The message is processed twice — the idempotency check (Implementation Step 5.6 / Step 8.1) is missing or the dedup key is wrong.

Test 6 — Catch-up recovers a missed event

Do: Temporarily delete the subscription (`Remove-MgSubscription`), send a message to `svc-mailevents@contoso.com` while there is no subscription, then run `wf-SubscriptionRenewal` (which recreates the subscription and runs the reconcile read). **Expected — PASS:** The message sent during the gap is picked up by the **catch-up read** and processed once; the subscription is recreated and validated; the watermark advances. **FAIL:** The gap message is never processed — the reconcile/watermark logic (Implementation Step 7.3) is missing or the watermark did not cover the gap.

Test 7 — Least-privilege scoping holds

Do: `Test-ApplicationAccessPolicy -AppId '<mi app id>' -Identity svc-mailevents@contoso.com` and `-Identity someone-else@contoso.com`. **Expected — PASS: Granted** for the target mailbox, **Denied** for the control mailbox — the app cannot read other mailboxes. (Depth in product #4.) **FAIL:** The control mailbox returns Granted — scoping is not applied. See product #4.

Pass/fail gate

Gate	Requirement
Receiver pass	Tests 1–3 pass — handshake creates the subscription, a real message fires and is read, a forged call is rejected
Durability pass	Tests 4–6 pass — renewal extends, dedup blocks doubles, catch-up recovers a gap
Security pass	Test 7 passes — access is scoped to the one mailbox before expanding
Expand pass	Tests 1–2, 4, 7 pass for each additional mailbox before the next expansion

6. Troubleshooting Guide — Event-Driven Mail Processing with Microsoft Graph Change Notifications

Use `GRAPHSUB-Discovery.ps1` / `GRAPHSUB-Validate.ps1` for readiness, `GRAPHSUB-Report.ps1` for the subscription inventory + expiry, `GRAPHSUB-Test-Notification.ps1` for coverage evidence, and `GRAPHSUB-Renew.ps1` (defaults to `-WhatIf`) to extend or recreate a subscription. Objects are referenced by their names — `wf-NotificationReceiver`, `wf-SubscriptionRenewal`, `svc-mailevents@contoso.com`, `MailEvents-ClientState`. **Every Graph limit/name below is version-specific — verify against current Microsoft documentation before relying on it.**

A. Subscription creation fails

Symptom	Cause	Remediation
`New-MgSubscription` returns "subscription validation request failed" / "endpoint validation failed"	The receiver did not echo the `validationToken` correctly or in time	See section B — fix the handshake branch, then recreate
"expirationDateTime is too far in the future" (or similar)	Requested lifetime exceeds Graph's current max for a mail resource	Lower `expirationDateTime` to within the allowed window (verify the current max with Microsoft); `GRAPHSUB-Renew.ps1` uses a conservative default
"resource is invalid / not supported"	The `resource` string is malformed or that resource/`changeType` is not supported	Use `users/svc-mailevents@contoso.com/messages` with `changeType=created`; verify supported resources/changeTypes with Microsoft
403 / insufficient privileges creating the subscription	The identity creating the subscription lacks the required permission, or mailbox scoping denies it	Confirm the app permission is granted + consented (Step 2) and scoping grants the target mailbox (Step 3 / product #4); re-run `GRAPHSUB-Validate.ps1`

B. Validation handshake fails

• **The handshake must be first and fast.** `wf-NotificationReceiver` must, before anything else, detect the `validationToken` query parameter and return it as the response body with `Content-Type: text/plain` and HTTP 200, within Graph's timeout (verify the exact timeout with Microsoft).

• **Common mistakes:** returning JSON instead of `text/plain`; not URL-decoding the token; the workflow doing slow work (Key Vault reads, loops) *before* responding; the endpoint not publicly reachable.

• **Check:** from an external network, `curl -i "https://<wf-NotificationReceiver URL>&validationToken=test123"` should return `200` with body `test123` and `Content-Type: text/plain`.

• **Fix:** move the validation branch to the very top of the workflow; return the raw token; then recreate the subscription.

C. Notifications stop arriving (expired subscription)

Symptom	Cause	Remediation
Mail arrives but `wf-NotificationReceiver` no longer runs	The subscription **expired** and was not renewed	Run `GRAPHSUB-Report.ps1` (state = EXPIRED) → `GRAPHSUB-Renew.ps1 -Mailbox svc-mailevents@contoso.com -Execute` (or `-RecreateIfMissing`); then run the catch-up read to recover the gap
Renewal workflow errors	`wf-SubscriptionRenewal` cadence too close to expiry, or the identity lost permission	Shorten the recurrence to well within the max lifetime (verify); confirm the MI still holds the permission (`GRAPHSUB-Validate.ps1`)
Subscription silently disappears	Deleted, tenant policy, or lifecycle event	Have `wf-SubscriptionRenewal` **recreate** on missing (falls through to `New-MgSubscription`); subscribe to lifecycle notifications if supported (verify)
Mail was missed during the gap	No subscription while messages arrived	Run `GRAPHSUB-Test-Notification.ps1` to see messages that arrived while lapsed; the **catch-up read** (from the watermark) reprocesses them once

D. 403 when reading the changed message

• **Symptom:** the notification fires and `wf-NotificationReceiver` runs, but the `GET /users/svc-mailevents@contoso.com/messages/{id}` returns **403 Forbidden**.

• **Cause 1 — permission not granted/consented:** the managed identity does not hold the application permission (e.g. `Mail.Read`), or admin consent was not applied. **Check:** `GRAPHSUB-Validate.ps1`. **Fix:** Implementation Step 2.

• **Cause 2 — scoping denies the mailbox:** the Application Access Policy / Graph RBAC scope excludes `svc-mailevents@contoso.com`. **Check:** `Test-ApplicationAccessPolicy -AppId '<mi app id>' -Identity svc-mailevents@contoso.com` should return **Granted**. **Fix:** correct the scope group / RBAC assignment (product #4). Access-policy changes can take time to propagate.

• **Cause 3 — wrong identity/audience:** the HTTP action is not authenticating as the managed identity, or the token audience is not `https://graph.microsoft.com` (or the national-cloud equivalent). **Fix:** set Authentication = Managed identity with the correct audience.

E. Duplicate processing

Symptom	Cause	Remediation
The same message is processed more than once	Notifications are at-least-once; retries and overlapping catch-up reads re-deliver	Enforce idempotency: key on the message **id** (+ `changeKey`/internet message id), check the dedup store before processing, skip if seen (Implementation Step 5.6 / 8.1)
Duplicates spike after failures	Receiver returned non-2xx, so Graph retried	Acknowledge with **202 quickly**, then process asynchronously; make downstream actions safe to repeat
Catch-up read reprocesses live-notified messages	Watermark overlaps already-processed items	The dedup store must be shared between the receiver and the catch-up read so both consult the same seen-ids

F. clientState mismatch

• **Symptom (legitimate):** `wf-NotificationReceiver` logs "clientState mismatch — dropped" for real notifications. **Cause:** the value in `MailEvents-ClientState` no longer matches the `clientState` set on the subscription (e.g. the secret was rotated but the subscription was not recreated). **Fix:** align them — recreate the subscription with the current secret value (`GRAPHSUB-Renew.ps1 -RecreateIfMissing …`) or update the stored secret to match.

• **Symptom (expected/good):** a forged or stray POST is dropped for clientState mismatch. **This is correct** — the public endpoint is doing its job. Do not disable the check.

• **Never** process a notification whose `clientState` does not match; that is the only thing standing between your public URL and an attacker driving the workflow.

G. Scripts — Microsoft.Graph connection and matching

• **Connect with the right scopes.** Interactive runs need delegated scopes: `Subscription.Read.All` (discovery/report), `Application.Read.All` + `Directory.Read.All`

(permission checks), `Subscription.ReadWrite` (renew), `Mail.Read` (notification-evidence). Each script's header lists its scopes. **Verify the exact scope names against current Microsoft documentation** — they can change.

• **Subscriptions are owned by their creator.** `Get-MgSubscription` returns subscriptions owned by the **identity you authenticate as**. Subscriptions created by the **managed identity** may **not** appear in an interactive delegated session — cross-check the Logic App run history, or authenticate as the MI. The scripts flag this rather than assume "no subscription = broken."

• **Exact-name matching.** Scripts resolve the managed identity by **display name** (`logic-mailevents-receiver-pilot`) or **Object ID**; pass `-ManagedIdentityObjectId` when display names are ambiguous. The Graph resource SP is matched by the stable AppId `00000003-0000-0000-c000-000000000000`, not a display name.

• `-WhatIf` **is the default state-changer safety.** Only `GRAPHSUB-Renew.ps1` changes state, and it **defaults to** `-WhatIf` — it prints what it would do and changes nothing until you pass `-Execute`. Preview first, always.

• **National clouds.** Pass `-Environment USGov` / `USGovDoD` for sovereign tenants — the default `Global` targets commercial endpoints. **Verify the correct environment value and endpoints against current Microsoft documentation.**

H. When to escalate

If the subscription creates and validates, the identity holds the scoped permission (`GRAPHSUB-Validate.ps1` all PASS), `clientState` matches, and notifications still do not arrive for new mail after confirming an active subscription with `GRAPHSUB-Report.ps1`, collect the `wf-NotificationReceiver` run history, the subscription id/expiry, and the `GRAPHSUB-Report.ps1` CSV, and open a Microsoft support case.

7. Best Practices — Event-Driven Mail Processing with Microsoft Graph Change Notifications

• **Answer the validation handshake first, and fast.** Make the `validationToken` echo the very first thing `wf-NotificationReceiver` does, returning `text/plain`/200 within Graph's timeout. Everything else in the workflow comes after — a slow or wrong handshake means the subscription never gets created.

• **Always verify** `clientState`**.** Your `notificationUrl` is public; anyone can POST to it. Compare each notification's `clientState` to the secret in `MailEvents-ClientState` and drop anything that does not match. Never process an unverified call.

• **Treat delivery as at-least-once — be idempotent.** Notifications can arrive more than once or not at all. Key every message by its **id** (plus `changeKey`/internet message id), check a durable dedup store before acting, and design downstream actions to be safe to repeat. This is how you get *effective* exactly-once.

• **Renewal is not optional.** Subscriptions expire. Run `wf-SubscriptionRenewal` on a cadence **comfortably shorter than the current max lifetime** (verify the number with Microsoft), and have it **recreate** a subscription that has already lapsed, not just extend a live one.

• **Reconcile to cover gaps.** Pair the webhook with a periodic **catch-up read** from a stored watermark, so anything that arrives during a lapsed subscription or a dropped notification is still processed once. A pure push design with no reconcile *will* eventually lose an event.

• **Least privilege, one mailbox.** Grant the smallest application permission that lets the workflow read a message, and scope it to `svc-mailevents@contoso.com` only (Application Access Policy or Graph RBAC — verify which is current; depth in product #4). Never grant a write scope "to be safe," and never leave the app able to read every mailbox.

• **No stored credentials — managed identity only.** The whole point of this family is credential-free auth. Keep secrets to just the `clientState` value in Key Vault; never put a client secret or service-account password in the workflow.

• **Acknowledge quickly, process asynchronously.** Return 202 promptly so Graph does not retry, then do the real work after — fewer retries means fewer duplicates to dedup.

• **Monitor both workflows.** Alert on renewal failures and on a run of receiver failures; a silent renewal failure is the classic way notifications "just stop." Use Graph lifecycle notifications if supported (verify).

• **Never hard-code Graph limits.** Maximum lifetime, renewal cadence, supported resources, permission names, encrypted-payload rules, and national-cloud endpoints all change — read them from current Microsoft documentation, not from memory or a blog.

• **Use the push pattern only when a public endpoint exists.** If Graph cannot reach your endpoint, do not force it — use the **polling** pattern (product #3) instead. Pick the pattern the network allows.

8. Deliverables Summary — Event-Driven Mail Processing with Microsoft Graph Change Notifications

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What the change-notification (webhook) pattern is — subscription + receiver + managed identity; why event-driven over the O365 connector or polling; benefits, limitations; pointers to products #1/#3/#4
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (public HTTPS endpoint, managed identity, app permission + consent, mailbox, Microsoft.Graph, TLS, national cloud)
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable MI, grant + consent least-priv permission, scope to one mailbox, build the receiver (handshake → clientState → idempotent read), create the subscription, build renewal + catch-up, reliability, pilot

Document	Purpose
5. Testing Guide	Validation procedures with PASS/FAIL gates (handshake, real event, forged-call rejection, renewal, dedup, catch-up, scoping)
6. Troubleshooting Guide	Field issues and remediation (creation fails, handshake fails, notifications stop, 403 reading the message, duplicates, clientState mismatch)
7. Best Practices	Field-proven recommendations

PowerShell scripts (Microsoft.Graph)

Script	What it does
`GRAPHSUB-Discovery.ps1`	Read-only — lists current Graph change-notification subscriptions (resource, expiry, notificationUrl) and the managed identity's application-permission grants
`GRAPHSUB-Validate.ps1`	PASS/FAIL — managed identity + least-privilege permission present, an active non-expired subscription on the target mailbox exists, receiver reachable as far as Graph can be checked
`GRAPHSUB-Report.ps1`	Subscription inventory (resource, expiry, notificationUrl host) with a CSV export
`GRAPHSUB-Renew.ps1`	The only state-changer — renews/extends a subscription nearing expiry; **defaults to -WhatIf** and says so
`GRAPHSUB-Test-Notification.ps1`	Read-only helper — checks recent notification/processing evidence and flags a lapsed subscription (a gap in coverage)

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with an explicit end-to-end event test and a documented rollback (delete the subscription; the mailbox is untouched).

Outcome

Near-real-time mail processing driven by a Graph change-notification subscription and a managed identity — event-driven, credential-free, scoped to one mailbox, and operationally durable with renewal and catch-up built in — proven on a pilot mailbox before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GRAPHSUB-Discovery.ps1

Purpose: Read-only discovery for the "Event-Driven Mail Processing with Microsoft Graph Change Notifications" solution. Lists the Graph change-notification subscriptions visible to the identity you authenticate as (resource, changeType, expirationDateTime, notificationUrl), flags any that are expired or expiring soon, and — for the Logic App's managed identity — lists the Microsoft Graph application-permission (app-role) grants. Makes no changes. HOW TO RUN pwsh -File .\GRAPHSUB-Discovery.ps1 pwsh -File .\GRAPHSUB-Discovery.ps1 -ManagedIdentityDisplayName 'logic-mailevents-receiver-pilot' pwsh -File .\GRAPHSUB-Discovery.ps1 -ManagedIdentityObjectId '<mi object id>' GRAPH SCOPES (delegated, for interactive discovery) Subscription.Read.All — read change-notification subscriptions Application.Read.All — read the managed identity's service principal + app-role grants Directory.Read.All — resolve the Microsoft Graph resource service principal (Connect with: Connect-MgGraph -Scopes 'Subscription.Read.All','Application.Read.All','Directory.Read.All') EXPECTED RESULTS A snapshot of current subscriptions with expiry state (OK / EXPIRING / EXPIRED) and the managed identity's Graph application-permission grants — the readiness picture before you validate or renew. NOTE: Graph subscriptions are owned by the identity that created them; an interactive delegated session may not see subscriptions created by the managed identity — verify against current Microsoft documentation and cross-check the Logic App run history. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; creates, renews, and deletes nothing. Verify all Graph specifics (max lifetime, supported resources, permission names, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([string]$ManagedIdentityDisplayName = 'logic-mailevents-receiver-pilot', [string]$ManagedIdentityObjectId, [int]$ExpiringWithinHours = 24, [string]$Environment = 'Global' # 'Global' | 'USGov' | 'USGovDoD' — national clouds: verify with Microsoft)

How to run: pwsh -File .\GRAPHSUB-Discovery.ps1 pwsh -File .\GRAPHSUB-Discovery.ps1 -ManagedIdentityDisplayName 'logic-mailevents-receiver-pilot' pwsh -File .\GRAPHSUB-Discovery.ps1 -ManagedIdentityObjectId '<mi object id>' GRAPH SCOPES (delegated, for interactive discovery) Subscription.Read.All — read change-notification subscriptions Application.Read.All — read the managed identity's service principal + app-role grants Directory.Read.All — resolve the Microsoft Graph resource service principal (Connect with: Connect-MgGraph -Scopes 'Subscription.Read.All','Application.Read.All','Directory.Read.All') EXPECTED RESULTS A snapshot of current subscriptions with expiry state (OK / EXPIRING / EXPIRED) and the managed identity's Graph application-permission grants — the readiness picture before you validate or renew. NOTE: Graph subscriptions are owned by the identity that created them; an interactive delegated session may not see subscriptions created by the managed identity — verify against current Microsoft documentation and cross-check the Logic App run history. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; creates, renews, and deletes nothing. Verify all Graph specifics (max lifetime, supported resources, permission names, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([string]$ManagedIdentityDisplayName = 'logic-mailevents-receiver-pilot', [string]$ManagedIdentityObjectId, [int]$ExpiringWithinHours = 24, [string]$Environment = 'Global' # 'Global' | 'USGov' | 'USGovDoD' — national clouds: verify with Microsoft)

GRAPHSUB-Renew.ps1

Purpose: Safe renewal for the "Event-Driven Mail Processing with Microsoft Graph Change Notifications" solution. Finds the subscription on the target mailbox (or a subscription id you pass) and extends its expirationDateTime by a requested amount that must stay WITHIN Graph's current maximum lifetime for the resource. If the subscription has already expired/been deleted, it can optionally RECREATE it (requires -NotificationUrl and -ClientState). DEFAULTS TO -WhatIf — it prints the change it would make and changes nothing until you pass -Execute. This is the manual/operator companion to the wf-SubscriptionRenewal workflow. HOW TO RUN Preview (default, no change): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Renew.ps1 -SubscriptionId '<id>' -ExtendMinutes 4230 Apply for real (explicit): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' -Execute Recreate if already expired (explicit): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' -RecreateIfMissing ` -NotificationUrl '<wf-NotificationReceiver URL>' -ClientState '<MailEvents-ClientState value>' -Execute GRAPH SCOPES (delegated) Subscription.Read.All — find the subscription Subscription.ReadWrite — PATCH (extend) / POST (recreate) the subscription [verify exact scope name with Microsoft] (Connect-MgGraph -Scopes 'Subscription.ReadWrite','Subscription.Read.All') EXPECTED RESULTS In -WhatIf mode (default): a printed description of the extend (or recreate) — nothing changes. With -Execute: the subscription's expirationDateTime moves forward (or it is recreated) and the new value is printed. The requested -ExtendMinutes MUST be within Graph's current max lifetime for a mail resource — verify against current Microsoft documentation; an over-long value is rejected by Graph. .NOTES Azure Solutions — DEFAULTS TO -WhatIf. The repeatable way. The default -ExtendMinutes is a conservative placeholder; the true maximum is version-specific — verify against current Microsoft documentation and never hard-code a lifetime you have not checked. > [CmdletBinding()] param([string]$Mailbox, [string]$SubscriptionId, [int]$ExtendMinutes = 4230, # placeholder — VERIFY the current max lifetime with Microsoft [switch]$RecreateIfMissing, [string]$NotificationUrl, # required for recreate [string]$ClientState, # required for recreate [string]$Resource, # recreate: defaults to users/<Mailbox>/messages [string]$ChangeType = 'created', [string]$Environment = 'Global', [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Renew.ps1 -SubscriptionId '<id>' -ExtendMinutes 4230 Apply for real (explicit): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' -Execute Recreate if already expired (explicit): pwsh -File .\GRAPHSUB-Renew.ps1 -Mailbox 'svc-mailevents@contoso.com' -RecreateIfMissing ` -NotificationUrl '<wf-NotificationReceiver URL>' -ClientState '<MailEvents-ClientState value>' -Execute GRAPH SCOPES (delegated) Subscription.Read.All — find the subscription Subscription.ReadWrite — PATCH (extend) / POST (recreate) the subscription [verify exact scope name with Microsoft] (Connect-MgGraph -Scopes 'Subscription.ReadWrite','Subscription.Read.All') EXPECTED RESULTS In -WhatIf mode (default): a printed description of the extend (or recreate) — nothing changes. With -Execute: the subscription's expirationDateTime moves forward (or it is recreated) and the new value is printed. The requested -ExtendMinutes MUST be within Graph's current max lifetime for a mail resource — verify against current Microsoft documentation; an over-long value is rejected by Graph. .NOTES Azure Solutions — DEFAULTS TO -WhatIf. The repeatable way. The default -ExtendMinutes is a conservative placeholder; the true maximum is version-specific — verify against current Microsoft documentation and never hard-code a lifetime you have not checked. > [CmdletBinding()] param([string]$Mailbox, [string]$SubscriptionId, [int]$ExtendMinutes = 4230, # placeholder — VERIFY the current max lifetime with Microsoft [switch]$RecreateIfMissing, [string]$NotificationUrl, # required for recreate [string]$ClientState, # required for recreate [string]$Resource, # recreate: defaults to users/<Mailbox>/messages [string]$ChangeType = 'created', [string]$Environment = 'Global', [switch]$Execute # omit for -WhatIf

(default safe mode))

GRAPHSUB-Report.ps1

Purpose: Read-only reporting for the "Event-Driven Mail Processing with Microsoft Graph Change Notifications" solution. Produces a subscription inventory — resource, changeType, expirationDateTime, computed expiry state (OK / EXPIRING / EXPIRED), time-to-expiry, and the notificationUrl host (host only, not the full secret-bearing URL) — to the console and a timestamped CSV. Useful for a periodic coverage/expiry review across all monitored mailboxes. HOW TO RUN pwsh -File .\GRAPHSUB-Report.ps1 pwsh -File .\GRAPHSUB-Report.ps1 -OutputPath 'C:\Reports' -ExpiringWithinHours 24 GRAPH SCOPES (delegated, for interactive reporting) Subscription.Read.All (Connect-MgGraph -Scopes 'Subscription.Read.All') EXPECTED RESULTS A table and a CSV (GRAPHSUB-Subscriptions-<timestamp>.csv) of every visible subscription with its expiry state — the artefact you keep to prove coverage and catch a subscription drifting toward expiry. NOTE: only subscriptions owned by the authenticating identity are visible; the MI's own subscriptions may require authenticating as the MI — verify with Microsoft. .NOTES Azure Solutions — read-only. The repeatable way. Reports the notificationUrl HOST only (the full URL can carry a secret token). Verify Graph specifics against current Microsoft documentation. > [CmdletBinding()] param([string]$OutputPath = '.', [int]$ExpiringWithinHours = 24, [string]$Environment = 'Global')

How to run: pwsh -File .\GRAPHSUB-Report.ps1 pwsh -File .\GRAPHSUB-Report.ps1 -OutputPath 'C:\Reports' -ExpiringWithinHours 24 GRAPH SCOPES (delegated, for interactive reporting) Subscription.Read.All (Connect-MgGraph -Scopes 'Subscription.Read.All') EXPECTED RESULTS A table and a CSV (GRAPHSUB-Subscriptions-<timestamp>.csv) of every visible subscription with its expiry state — the artefact you keep to prove coverage and catch a subscription drifting toward expiry. NOTE: only subscriptions owned by the authenticating identity are visible; the MI's own subscriptions may require authenticating as the MI — verify with Microsoft. .NOTES Azure Solutions — read-only. The repeatable way. Reports the notificationUrl HOST only (the full URL can carry a secret token). Verify Graph specifics against current Microsoft documentation. > [CmdletBinding()] param([string]$OutputPath = '.', [int]$ExpiringWithinHours = 24, [string]$Environment = 'Global')

GRAPHSUB-Test-Notification.ps1

Purpose: Read-only coverage check for the "Event-Driven Mail Processing with Microsoft Graph Change Notifications" solution. Answers "is the webhook actually working right now?" without changing anything, by correlating two facts for the target mailbox: 1. Is there an ACTIVE, non-expired change-notification subscription on the mailbox? (If not, new mail is NOT being pushed — a coverage gap.) 2. Are there recent messages that arrived AFTER a reference time (default: last hour) that the event pipeline should have processed? (These are what you reconcile against the receiver's run history / dedup store to confirm nothing was missed.) It reports a lapsed subscription and lists recent messages as processing evidence; it does not create, renew, delete, or send anything. HOW TO RUN pwsh -File .\GRAPHSUB-Test-Notification.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Test-Notification.ps1 -Mailbox 'svc-mailevents@contoso.com' -SinceHours 4 GRAPH SCOPES (delegated) Subscription.Read.All — check the subscription is alive Mail.Read — read recent messages of the mailbox as processing evidence (least privilege; verify the current scope name with Microsoft) (Connect-MgGraph -Scopes 'Subscription.Read.All','Mail.Read') EXPECTED RESULTS A clear verdict: subscription ACTIVE or LAPSED (gap), plus a list of recent messages (received time, subject, id) to reconcile against what the receiver actually processed. A LAPSED subscription with recent messages = events were likely missed → recover via the catch-up read / GRAPHSUB-Renew.ps1. This checks for

EVIDENCE; it does not itself inject a test event. .NOTES Azure Solutions — read-only. The repeatable way. Reads message metadata only. Verify Graph specifics (supported resources, permission names, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$Mailbox, [int]$SinceHours = 1, [string]$Environment = 'Global')

How to run: pwsh -File .\GRAPHSUB-Test-Notification.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Test-Notification.ps1 -Mailbox 'svc-mailevents@contoso.com' -SinceHours 4 GRAPH SCOPES (delegated) Subscription.Read.All — check the subscription is alive Mail.Read — read recent messages of the mailbox as processing evidence (least privilege; verify the current scope name with Microsoft) (Connect-MgGraph -Scopes 'Subscription.Read.All','Mail.Read') EXPECTED RESULTS A clear verdict: subscription ACTIVE or LAPSED (gap), plus a list of recent messages (received time, subject, id) to reconcile against what the receiver actually processed. A LAPSED subscription with recent messages = events were likely missed → recover via the catch-up read / GRAPHSUB-Renew.ps1. This checks for EVIDENCE; it does not itself inject a test event. .NOTES Azure Solutions — read-only. The repeatable way. Reads message metadata only. Verify Graph specifics (supported resources, permission names, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$Mailbox, [int]$SinceHours = 1, [string]$Environment = 'Global')

GRAPHSUB-Validate.ps1

Purpose: Read-only PASS/FAIL validation for the "Event-Driven Mail Processing with Microsoft Graph Change Notifications" solution. Checks, for the target mailbox: 1. the managed identity exists and holds a least-privilege Graph application permission (default Mail.Read — verify the current least-priv scope with Microsoft); 2. an ACTIVE, non-expired change-notification subscription exists on the mailbox's message resource; 3. the receiver endpoint (the subscription's notificationUrl) is reachable over HTTPS as far as this script can check (a best-effort GET — Graph's own validation is authoritative). Prints PASS/FAIL per check and an overall gate. Makes no changes. HOW TO RUN pwsh -File .\GRAPHSUB-Validate.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Validate.ps1 -Mailbox 'svc-mailevents@contoso.com' -LeastPrivPermission 'Mail.Read' -ManagedIdentityDisplayName 'logic-mailevents-receiver-pilot' GRAPH SCOPES (delegated, for interactive validation) Subscription.Read.All, Application.Read.All, Directory.Read.All (Connect-MgGraph -Scopes 'Subscription.Read.All','Application.Read.All','Directory.Read.All') EXPECTED RESULTS PASS on all three checks = the pattern is wired up (identity + least-priv permission, a live subscription on the mailbox, a reachable endpoint). Any FAIL points at the exact gap. NOTE: endpoint reachability here is best-effort; Graph's creation-time handshake is authoritative. .NOTES Azure Solutions — read-only. The repeatable way. Verify all Graph specifics (max lifetime, supported resources, least-privilege permission name, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$Mailbox, [string]$LeastPrivPermission = 'Mail.Read', [string]$ManagedIdentityDisplayName = 'logic-mailevents-receiver-pilot', [string]$ManagedIdentityObjectId, [string]$Environment = 'Global')

How to run: pwsh -File .\GRAPHSUB-Validate.ps1 -Mailbox 'svc-mailevents@contoso.com' pwsh -File .\GRAPHSUB-Validate.ps1 -Mailbox 'svc-mailevents@contoso.com' -LeastPrivPermission 'Mail.Read' -ManagedIdentityDisplayName 'logic-mailevents-receiver-pilot' GRAPH SCOPES (delegated, for interactive validation) Subscription.Read.All, Application.Read.All, Directory.Read.All (Connect-MgGraph -Scopes 'Subscription.Read.All','Application.Read.All','Directory.Read.All') EXPECTED RESULTS PASS on all three checks = the pattern is wired up (identity + least-priv permission, a live subscription on the mailbox, a reachable endpoint). Any FAIL points at the exact gap. NOTE: endpoint reachability here is best-effort; Graph's creation-time handshake is authoritative. .NOTES Azure Solutions — read-only. The

repeatable way. Verify all Graph specifics (max lifetime, supported resources, least-privilege permission name, national-cloud endpoints) against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$Mailbox, [string]$LeastPrivPermission = 'Mail.Read', [string]$ManagedIdentityDisplayName = 'logic-mailevents-receiver-pilot', [string]$ManagedIdentityObjectId, [string]$Environment = 'Global')